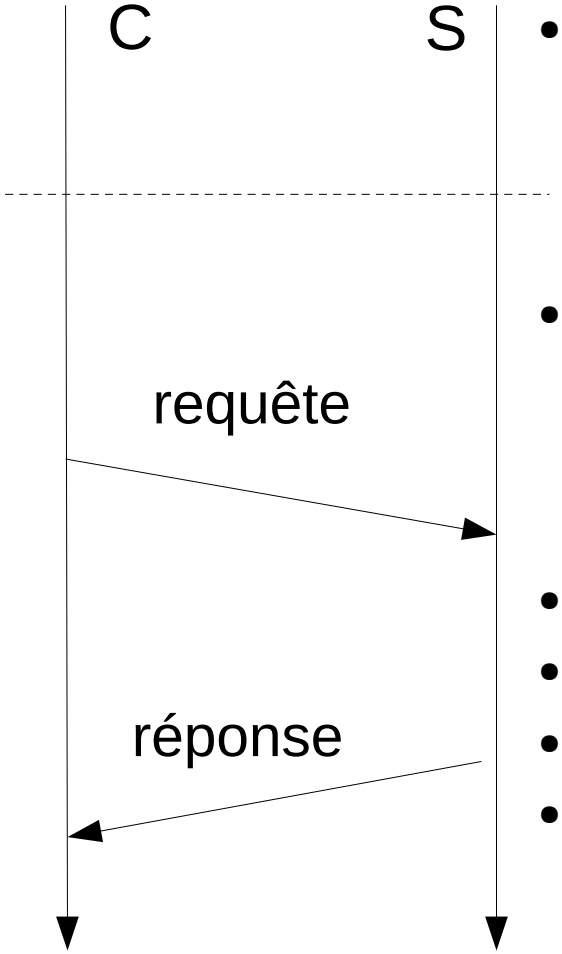
Généralités

- Application P2P =>
  - client et serveur dans la même application
  - mais ici, un seul des 2 exécuté, au choix de l’utilisateur (en fonction de la tâche)

« Algo. » C/S (diagramme de séquences)

- Création socket
- Demande de connexion (en même temps que création en Java)
- Construction requête
- Envoi requête (write)
- Attente (read) …



- Création socket S1
- Attente (accept, S1) …

d'une demande de connexion, d'un client

- Attente (read, S2) …

réception requête

- Analyse requête
- Construction réponse
- Envoi réponse (write, S2)
- Fermeture sockets (les 2)

réception réponse

- Utilisation réponse
- Fermeture socket

éventuellement en boucle

Échanges TCP, IP, … pas montrés !

Socket Java – Client « écho », socket en mode connecté (i.e. avec TCP)

```
import java.net.Socket,   java.net.InetAddress,    java.io.* ;


public class client {
    public static void main(String args[])     {
        try {
        // coord. du serveur à fournir « manuellement » :
            int port = xxxxx;
            InetAddress adr = InetAddress.getByName("w.x.y.z");
       // création socket et demande connexion :
            Socket s = new Socket(adr, port);
```

Socket Java – Client « écho », socket en mode connecté (i.e. avec TCP)

- byte[] envoi = ...; // requête à envoyer

// « sortie » et « entree » sont les flux associés à la socket :

- OutputStream sortie = s.getOutputStream();
- sortie.write(envoi);
- int taille_tampon = 10; // !!! attention à cette valeur !!!
- InputStream entree = s.getInputStream();
- byte[] recu = new byte[taille_tampon];
- entree.read(recu);

// bloquant jusqu'à réception réponse

Socket Java – Client « écho », socket en mode connecté (i.e. avec TCP)

```
            s.close();  // fermeture connexion
        }
        catch (IOException e) {
    // exceptions à gérer (y'en a d'autres, sur les flux aussi)
            e.printStackTrace();
        }
    }
}
```

Socket Java – Serveur « echo », socket en mode connecté (i.e. avec TCP)

- import java.net.ServerSocket, java.net.Socket, java.net.InetAddress, java.io.*;

- public class client {
- public static void main(String args[]) {
- try {

// coord. du serveur à fixer :

- int port = <port>; // en prévoir 1 par défaut, modifiable
- InetAddress adr = InetAddress.getByName("<IP>");

// détection automatique incertaine => modifiable

Socket Java – Serveur « echo », socket en mode connecté (i.e. avec TCP)

// Création socket :

- ServerSocket s = new ServerSocket(port, 10, adr);

// attente d'un client (bloquant jusqu'à réception d'une demande de connexion) :

- Socket c = s.accept();

// création nouvelle socket !!!

Socket Java – Serveur « echo », socket en mode connecté (i.e. avec TCP)

// flux associés à socket c, pas s

// on attend une requête, puis on la traite, puis on envoie la réponse :

- InputStream entree = c.getInputStream();
- int taille_tampon = 10; // !!! attention valeur
- byte[] recu = new byte[taille_tampon];
- entree.read(recu); // bloquant jusqu'à réception requête
- OutputStream sortie = c.getOutputStream();
- byte[] envoi = recu; // envoi réponse = requête ici (écho)
- sortie.write(envoi);

Socket Java – Serveur « echo », socket en mode connecté (i.e. avec TCP)

```
    // 2 sockets à fermer :
            c.close();
            s.close();
        }
        catch (IOException e) {
            e.printStackTrace();
    // exceptions à gérer, parfois différentes de celles du client
        }
    }
}
```

Socket Java – Récupération automatique d'IP de l'hôte pour le serveur

- 3 possibilités : https://www.baeldung.com/java-get-ip-address
- Aucune solution fiable ; privilégier un serveur IUT plutôt que Google
- Utiliser la méthode getLocalHost() d'un objet InetAddress
- Au niveau du serveur mais en tant que client, se « connecter » à un serveur connu et utiliser la méthode getLocalAddress() de l'objet socket utilisé
- Énumérer les IP de chaque interface réseau (objet NetworkInterface) ; pour avoir une liste déroulante

  https://docs.oracle.com/javase/tutorial/networking/nifs/listing.html

Socket Java – Remarques sur les flux

- Par défaut, échanges en octets (tableau de bytes)
- Possibilité d’utiliser des classes de plus haut niveau mais attention aux conversions !

JavaFX et threads

- « Rappel » thread : fil d'exécution « indépendant », « sans » données propres (partagées dans le processus parent)
- L'interface graphique d'une appli JavaFX ne fonctionne que dans un seul thread : le JavaFX Application Thread (ou UI thread)
- « Pour garantir la synchronisation et l'intégrité des composants graphiques »
- Si une méthode accapare les ressources, l'IHM est bloquée tant que la méthode n'est pas terminée
- => les opérations lourdes et/ou longues (comme les échanges réseaux) doivent être faites dans un (des) autre(s) thread(s), en arrière plan

JavaFX et threads

- Utiliser la librairie dédiée à la concurrence (au multitâche) sous JavaFX, notamment la classe Task

  https://docs.oracle.com/javafx/2/threads/jfxpub-threads.htm

- Ne pas utiliser les threads Java classiques car pas sûrs avec JavaFX. Pour information :
  - https://www.geeksforgeeks.org/java/java-threads/
  - https://www.jmdoudoux.fr/java/dej/chap-threads.htm
- Ne pas utiliser Platform.runLater() (déprécié)

JavaFX et threads - Exemple

- import javafx.application.Application; import javafx.event.ActionEvent; import javafx.event.EventHandler; import javafx.scene.Scene; import javafx.scene.control.Button; import javafx.stage.Stage; import javafx.scene.control.TextField; import javafx.scene.layout.GridPane; import javafx.geometry.Pos; import javafx.concurrent.Task;

- public class appli_fx extends Application {
- public static void main(String[] args) {
- launch(args);
- }
-

JavaFX et threads - Exemple

- @Override
- public void start(Stage primaryStage) throws Exception {
- primaryStage.setTitle("Test Task");
-
- TextField zoneTxt = new TextField();

// zone de texte quelconque utilisé dans cet exemple pour recueillir des infos, résultats d'autre(s) thread(s)

JavaFX et threads - Exemple

```
Task<Integer> tache = new Task<Integer>(){
// création task ; type à adapter
        @Override protected Integer call() throws Exception {
            updateMessage("tache lancee"); // MàJ UI
            for (int i=0;i<10;i++) {
                Thread.sleep(1000);
                if (isCancelled()){ break; }  // surveillance arrêt
                updateMessage("etat avancement : " + i);
            }
            return(0);          }
```

JavaFX et threads - Exemple

- // méthodes pour agir, suivre la tâche ; il en existe d’autres
- @Override protected void cancelled() {
- super.cancelled();
- updateMessage("tache arretee"); }
-
- @Override protected void succeeded() {
- super.succeeded();
- updateMessage("tache terminee"); }
- }; // fin définition Task

JavaFX et threads - Exemple

```
// pour mettre à jour la zone de texte « zoneTxt »
        zoneTxt.textProperty().bind(tache.messageProperty());

        Button btn1 = new Button(); // bouton pour lancer la tâche
        btn1.setText("Action");
        btn1.setOnAction(new EventHandler<ActionEvent>() {
            @Override public void handle(ActionEvent event) {
                new Thread(tache).start(); // lancement task
            }
        });
```

JavaFX et threads - Exemple

// bouton pour arrêter la tâche en cours

```
Button btn2 = new Button();
btn2.setText("Stop action");
btn2.setOnAction(new EventHandler<ActionEvent>() {
    @Override public void handle(ActionEvent event) {
        tache.cancel();
    }
});
```

JavaFX et threads - Exemple

```
// création de l'UI
        GridPane gp = new GridPane();
        gp.setAlignment(Pos.CENTER);
        gp.add(zoneTxt,0,0);
        gp.add(btn1,0,1);
        gp.add(btn2,0,2);
        primaryStage.setScene(new Scene(gp, 250, 250));
        primaryStage.show();
    }
}
```